

Неделя 2. Лекция 2

Внутренности агентов: context management, memory management, planning и tool orchestration

Аккуратный конспект по транскрибации с исправлением ошибок распознавания, техническими уточнениями и практическим блоком для задания недели.

Источник: транскрибация файла Nueva grabacion 4m4a.txt. Текст ниже не является дословной расшифровкой: повторы, мусор распознавания и разговорные отступления очищены; спорные места помечены как уточнения.

Содержание

- 1. Главная идея лекции
- 2. Что было исправлено и уточнено относительно транскрибации
- 3. Из чего состоит внутренняя работа агента
- 4. Context management: как управлять контекстом
- 5. Memory management: как устроена память агента
- 6. Planning: как агент превращает цель в план
- 7. Tool orchestration: как агент работает с инструментами
- 8. Практические лайфхаки при работе с агентами и code-assistant инструментами
- 9. Промпты: simple prompt, pro prompt и prompt pro max
- 10. JSON и TOON: зачем нужны token-efficient форматы
- 11. Цель недели и вариант реализации задания
- 12. Чек-лист для README и видео-демо
- 13. Готовые промпы для Cursor / Claude Code

1. Главная идея лекции

Вторая неделя посвящена инструментарию и внутреннему устройству агентов. Если первая неделя была про базовые LLM, промптинг и параметры генерации, то здесь фокус смещается на то, как агент удерживает контекст, как он хранит память, как строит план действий и как вызывает инструменты.

Главный практический тезис лекции: разработчик должен делегировать агентам не отдельные микродействия, а законченные этапы работы. Для этого агенту нужен управляемый контекст, понятная память, планирование и механизм работы с tools.

Внутренняя работа агента в рамках этой лекции сводится к четырём большим блокам:

- Context management - управление тем, что модель видит прямо сейчас в контекстном окне.
- Memory management - управление тем, что сохраняется между сессиями и как затем возвращается в контекст.
- Planning - превращение цели в план, задачи, стратегию исполнения и перепланирование.
- Tool orchestration - выбор инструментов, вызов tools, интерпретация результата, обработка ошибок и обновление состояния.

2. Что было исправлено и уточнено относительно транскрибации

В транскрибации много ошибок распознавания и несколько мест, которые в учебном конспекте лучше формулировать технически точнее.

В транскрибации / риск	Как правильно в конспекте
“лоломки”, “нейронки как угодно”	LLM / большие языковые модели. В тексте используется термин LLM, а разговорные варианты убраны.
“викторизируйте”	Векторизируете: превращаете текст/факты/документы в embeddings для последующего поиска и retrieval.
“Тор-В”	top_p. Вместе с top_k это параметры sampling, то есть выбора следующего токена после того, как модель уже посчитала вероятности.
“веса перебалансируются внутри сессии”	Точнее: веса модели во время обычного inference не меняются. Меняется распределение вероятностей ответа из-за нового контекста, истории диалога, системных инструкций и параметров генерации.
“у нейросети нет памяти”	Базовая LLM сама по себе не имеет постоянной памяти между независимыми запросами. Но продукт поверх LLM может иметь внешнюю память: файлы, БД, векторное хранилище, profile memory, summary и т.д.
“top_p/top_k сужают бесконечную базу знаний”	Лучше так не говорить. Роль, инструкции, RAG и выбранный контекст сужают предметную область. top_p/top_k управляют sampling на уровне следующего токена.
“из embeddings обратно получать текст”	В классическом RAG embeddings обычно не декодируют обратно в текст. Обычно хранят embeddings + ссылку на оригинальный текстовый chunk, находят близкие chunks и подмешивают их в prompt.
“Туне / Tune / Toone”	Судя по описанию, речь о TOON - Token-Oriented Object Notation: компактном формате для передачи структурированных данных в LLM-промптах.
“TOON всегда повышает точность на 30-60%”	Осторожнее: TOON может экономить токены, особенно на однотипных массивах объектов. Эффект по точности зависит от задачи, модели и необходимости строгого structured output.
“MCP раскроется позже”	Tool orchestration здесь даётся обзорно; MCP стоит воспринимать как один из способов стандартизировать подключение tools к агенту, а не как единственный вариант.

3. Из чего состоит внутренняя работа агента

На прошлой неделе уже была базовая цепочка агентного поведения: perception -> memory -> reasoning -> planning -> action -> feedback. Во второй лекции эта схема уточняется через инженерные компоненты, которые нужно реализовать или хотя бы понимать при работе с агентом.

```
perception -> memory -> reasoning -> planning -> action -> feedback
```

Инженерные блоки вокруг этой цепочки:

1. context management
2. memory management
3. planning
4. tool orchestration

3.1. Context management

Context management отвечает за то, что модель видит в текущем запросе. LLM не “помнит” весь мир и не держит в голове весь проект. Она получает ограниченный набор токенов: system prompt, user messages, assistant messages, подмешанные документы, результаты tools, summaries и т.д.

Если контекст собран плохо, агент начинает делать странные выводы: путает стек, пишет не те файлы, игнорирует ограничения, повторяет уже сделанное или ломает соседнюю часть проекта.

3.2. Memory management

Memory management отвечает за сохранение полезной информации между шагами, сессиями и задачами. Это не “магическая память модели”, а конкретная инженерная система: summary, JSON-файлы, база данных, vector store, правила, preferences, project state и другие структуры.

3.3. Planning

Planning превращает намерение пользователя в рабочий план. Хороший агент не должен сразу бросаться писать код. Сначала он должен понять цель, разбить её на задачи, выбрать стратегию, определить зависимости, при необходимости согласовать план с пользователем, а затем выполнять.

3.4. Tool orchestration

Tool orchestration отвечает за взаимодействие агента с внешним миром: файловой системой, терминалом, API, браузером, MCP-серверами, базой данных, тестами, линтерами и другими инструментами. Это слой, где агент перестаёт быть просто чат-ботом и становится рабочей системой.

4. Context management: как управлять контекстом

4.1. Почему контекст критичен

Базовая LLM не знает, что именно вы строите, пока вы ей это не сообщили. Да, модель обучалась на огромном корпусе данных, но конкретный проект, стек, архитектурные решения, правила репозитория и текущий статус работы должны быть явно поданы в контекст.

Современные коммерческие ассистенты частично делают это за пользователя: индексируют проект, подтягивают историю, хранят preferences, подмешивают правила. Но если вы строите собственного агента или настраиваете code-assistant, эту механику нужно понимать и контролировать.

4.2. Роли и инструкции

Роли и инструкции задают рамку поведения агента: кто он, какую задачу решает, какие ограничения обязан соблюдать и в каком формате отвечать. Это не замена контексту, но сильный способ сузить пространство решений.

Плохая рамка:
Напиши код.

Нормальная рамка:
Ты senior Python backend engineer. Работаешь в проекте FastAPI + SQLAlchemy 2.0 + PostgreSQL. Соблюдай текущую архитектуру, не меняй публичные контракты без необходимости, добавляй тесты, перед изменениями коротко составь план и явно перечисли файлы, которые будешь трогать.

4.3. Способы управления контекстом

Техника	Суть	Когда полезно	Риск
History compression	Сжать длинную историю в summary и использовать его как новый стартовый контекст.	Когда контекстное окно почти заполнено и нужно продолжать диалог.	Теряются детали; summary может быть неполным или ошибочным.

Техника	Суть	Когда полезно	Риск
Sliding window	Оставлять только последние N сообщений, постепенно выкидывая старые.	Когда разговор идёт вокруг одной темы и старые детали уже не важны.	Плохо работает, если в одном чате смешаны разные задачи.
Vector embeddings / retrieval	Векторизовать документы, факты или chunks и подмешивать только релевантные куски.	Для больших баз знаний, документации, проектных файлов.	Retrieval может найти не те chunks; найденные факты не гарантируют правильный вывод.
Context layers	Разделить контекст на стратегический, оперативный и task-level слой.	Для больших проектов, где разным агентам нужны разные части информации.	Сложно правильно разложить знания по слоям.
Clear / recreate dialog	Очистить контекст и начать новую задачу с чистого системного промпта.	Когда задача принципиально новая или чат загрязнён.	Можно потерять важные договорённости, если их не перенести явно.
Token pruning	Удалять мусорные или малозначимые фрагменты из контекста.	Для экономии токенов в длинных агентных циклах.	Слишком агрессивная обрезка может удалить важные нюансы.
Topic branches	Разделять разные темы на отдельные ветки/субконтексты.	Когда в одном проекте параллельно идут backend, frontend, tests, docs.	Нужна оркестрация, чтобы ветки не противоречили друг другу.

4.4. Практическое правило

Если агент делает странные вещи, первым делом проверяйте не модель, а контекст. Очень часто проблема не в “тупой нейронке”, а в том, что вы дали ей грязный, слишком широкий или противоречивый context.

Хороший запрос обычно содержит: язык, стек, цель, ограничения, релевантные файлы, критерии готовности, формат результата и правила проверки.

5. Memory management: как устроена память агента

5.1. Три уровня памяти

Тип памяти	Что это значит	Пример
No memory	Каждый запрос независим. Модель ничего не знает, кроме текущего prompt.	Одноразовый REST-запрос к LLM без истории.
Session memory	История внутри текущего диалога или agent run.	Модель видит предыдущие сообщения текущего чата.
Long-term memory	Данные сохраняются вне модели и подгружаются в будущие сессии.	Файлы состояния, user profile, vector DB, project memory, summaries.

5.2. Компрессированная память

Компрессированная память - это сохранение не всей истории, а её краткой свёртки: summary последнего этапа, summary дня, summary задачи или summary проекта.

- Плюсы: экономит токены; позволяет держать длинную историю; легко реализуется.
- Минусы: теряет детали; summary делает LLM, поэтому возможны ошибки; потерянные детали потом провоцируют галлюцинации.

- Практика: summary хорошо подходит для статуса проекта, но плохо подходит для точных требований, API-контрактов, edge cases и чисел.

5.3. Vector-based memory

Vector-based memory хранит факты, документы или chunks в виде embeddings и позволяет искать релевантные элементы по смысловой близости. Важно: обычно embeddings не превращают обратно в текст. На практике рядом с embedding хранится ссылка на оригинальный текстовый chunk, который затем и подмешивается в prompt.

- Плюсы: масштабируется; позволяет работать с большой базой знаний; вытаскивает релевантные фрагменты вместо всего контекста.

- Минусы: нужен отдельный embedding step; retrieval может ошибиться; найденные факты не гарантируют правильный reasoning.

- Практика: подходит для документации, базы знаний, больших репозиторий, истории задач.

5.4. Слоистая память

Слоистая память сохраняет знания по уровням приоритета. Например: общий контекст проекта, контекст текущей фичи, контекст конкретной задачи, личные preferences пользователя, запреты, архитектурные решения.

Пример слоёв памяти:

- L1. Global project rules: стек, архитектура, стиль кода
- L2. Feature context: что именно строим на этой неделе
- L3. Task context: конкретная задача текущего agent run
- L4. Temporary state: что уже сделано, какие ошибки были, какие файлы изменены

Это мощный подход для больших проектов, но его сложно реализовать: нужно правильно классифицировать память, обновлять её и решать, какие слои подмешивать в конкретный запрос.

5.5. Rule-based memory

Rule-based memory хранит правила в явном виде. Это ближе к экспертным системам и fuzzy logic, чем к чистому LLM-подходу. Пример правила: “если пользователь просит изменить публичный API, сначала предложи план миграции и не ломай обратную совместимость без подтверждения”.

- Плюсы: высокий контроль; больше детерминированности; удобно для hard constraints.

- Минусы: негибко; правила нужно поддерживать; всё, что вне правил, может быть обработано плохо.

- Практика: храните в правилах запреты, обязательные проверки, стиль проекта и требования безопасности.

5.6. Self-reflective / semantic memory

Self-reflective memory сохраняет не только факты, но и смыслы: стиль пользователя, предпочтения, устойчивые паттерны, тон, типичные задачи, долгосрочные цели. Благодаря этому ассистент кажется “узнающим” пользователя.

- Плюсы: повышает качество персонализации; позволяет агенту лучше подстраиваться под пользователя.

- Минусы: требует фильтрации; может закрепить неверные интерпретации; смыслы меняются со временем.
- Практика: полезно для персональных ассистентов и долгих проектов, но опасно без механизма обновления и удаления памяти.

5.7. Комбинации памяти

Уровень агента	Что обычно есть	Как выглядит поведение
Basic assistant	Текущий контекст + история сессии.	Отвечает на текущий запрос, но быстро теряет детали при длинном чате.
Smart assistant	Session memory + summaries + rules + иногда retrieval.	Лучше держит задачу, помнит часть предпочтений, меньше путается.
Advanced agent	Layered memory + vector retrieval + self-reflection + explicit state.	Похоже на устойчивую рабочую систему: помнит проект, план, ошибки, состояние и цели.

6. Planning: как агент превращает цель в план

Planning - это стадия, на которой агент не просто отвечает, а организует работу. Для code-assistant и автономных агентов это критично: без планирования модель начинает хаотично генерировать код и часто ломает уже работающие части.

Минимальный planning loop:

1. Определить цель.
2. Уточнить ограничения.
3. Разбить цель на задачи.
4. Выбрать стратегию исполнения.
5. Проверить, что стратегия подходит.
6. При необходимости перепланировать.
7. Подтвердить план у пользователя или автоматически перейти к выполнению.
8. После каждого шага обновлять state.

6.1. Что должен содержать хороший план

- Цель: какой результат должен быть на выходе.
- Файлы / зоны проекта: что потенциально будет изменено.
- Порядок шагов: сначала конфигурация, потом код, потом тесты, потом README и demo.
- Критерии готовности: как понять, что задача сделана.
- Проверки: какие команды запустить, какие кейсы протестировать.
- Rollback / риск: что может сломаться и как это контролировать.

6.2. Когда нужно перепланирование

Перепланирование нужно, когда выбранная стратегия не ложится на задачу: tool вернул ошибку, API недоступен, тесты показали архитектурную проблему, требований оказалось больше, чем ожидалось, или агент понял, что выбранный путь создаёт лишнюю сложность.

Хороший агент не должен "долбить стену". Он должен уметь остановиться, записать проблему в state, предложить новый план и продолжить уже по нему.

7. Tool orchestration: как агент работает с инструментами

Tool orchestration - это слой, где агент выбирает и вызывает инструменты. В лекции эта тема дана обзорно, потому что подробно она будет раскрыта на неделе MCP. Но базовую механику нужно понимать уже сейчас.

7.1. Цикл работы с tool

1. Tool selection: выбрать нужный инструмент.
2. Tool invocation: вызвать инструмент с корректными аргументами.
3. Result interpretation: понять, что вернул инструмент.
4. Error handling: обработать ошибку или неожиданный результат.
5. State update: записать, что произошло, и перейти к следующему шагу плана.

7.2. Почему tools сложнее, чем кажется

- Tool может вернуть ошибку API, timeout, неполные данные или неожиданный формат.
- Успешный HTTP-ответ не всегда означает, что цель достигнута.
- Агент должен проверять, тот ли tool он выбрал.
- Результат tool нужно интерпретировать в контексте плана, а не просто вставлять в ответ.
- После каждого tool-вызова нужно обновлять state, иначе агент потеряет ход выполнения.

7.3. Пример

Цель: узнать погоду и дать рекомендацию пользователю.

Плохой агент:

- вызвал weather API;
- вставил сырой JSON в ответ.

Нормальный агент:

- выбрал weather tool;
- вызвал его с городом и датой;
- проверил, что ответ валиден;
- извлёк температуру, осадки, ветер;
- обновил state: weather_checked=true;
- дал пользователю рекомендацию с учётом цели.

8. Практические лайфхаки при работе с агентами и code-assistant

8.1. Сужайте контекст

Это главный практический совет лекции. Указывайте язык, стек, задачу, файлы, ограничения, ожидаемый формат ответа и критерии готовности. Не заставляйте агента плавать в “мировой базе знаний”.

8.2. Индексируйте проект

Индексация и векторизация проекта помогают агенту находить релевантные файлы и связи. Это особенно важно в больших репозиториях, где ручное добавление всех файлов в prompt быстро забивает контекстное окно.

8.3. Документируйте проект

Минимальные docstrings, комментарии, README, architecture notes и правила проекта помогают агенту ориентироваться. Иногда достаточно дать модели сигнатуры функций и комментарии, а не весь исходный код. Это экономит контекст.

8.4. Делайте stage-based execution

Разбивайте работу на стадии и, если возможно, передавайте каждую стадию новому агенту или новому чистому контексту. Это позволяет не тащить весь мусор предыдущих шагов и снижает вероятность галлюцинаций.

8.5. Избегайте лишней суммаризации

History compression и summary полезны, но это “неизбежное зло”. Они экономят токены, но убивают детали. Если можно сохранить точный state, список решений, список изменённых файлов и тестовый результат - лучше сохранить это явно, а не только в виде summary.

9. Промпты: simple prompt, pro prompt и prompt pro max

9.1. Simple prompt

Напиши функцию на Kotlin, которая проверяет, является ли число простым.

Проблема: не указаны платформа, версия языка, ограничения, формат ответа, тесты, edge cases, производительность и контекст проекта. Модель что-то напишет, но это будет случайный generic-ответ.

9.2. Pro prompt

Сделай модуль для Android-приложения на Kotlin, который выполняет аутентификацию через Google OAuth.
Используй современный стек Android, добавь обработку ошибок, опиши зависимости и покажи пример интеграции.

Такой prompt уже лучше: есть экосистема, задача, протокол, примерный стек и ожидаемые части результата. Модель меньше угадывает и больше работает в нужной области.

9.3. Prompt pro max / metaprompt

Prompt pro max - это большой метапромпт, где заранее описаны роль, стек, архитектура, ограничения, формат ответа, критерии готовности, тесты, документация, ограничения по изменению файлов и стиль работы.

Пример структуры metaprompt:

ROLE:
You are a senior Python backend engineer.

PROJECT CONTEXT:
FastAPI + SQLAlchemy 2.0 + PostgreSQL. The project follows service/repository layering.

TASK:
Implement ...

CONSTRAINTS:
- Do not change public API without explanation.
- Add tests for success and failure cases.
- Keep backward compatibility.

PROCESS:

1. Inspect relevant files.
2. Create a short plan.
3. Implement changes.
4. Run tests.
5. Update README/demo notes.

OUTPUT FORMAT:

- Plan
- Changed files
- How to run
- Test results
- Risks

Большой prompt не всегда “жрёт слишком много контекста”. Если он экономит десятки уточняющих сообщений и снижает шум assistant-ответов, он может быть выгоднее, чем серия коротких мутных запросов.

10. JSON и TOON: зачем нужны token-efficient форматы

В лекции под “Туне” почти наверняка имеется в виду TOON - Token-Oriented Object Notation. Это компактное представление JSON-like данных для LLM-промптов. Его идея: не повторять одни и те же ключи в каждом объекте массива, а один раз объявить схему и дальше передавать строки значений.

10.1. Почему JSON дорогой для LLM

JSON удобен и является стандартом для API, но в prompt он может быть многословным: ключи, кавычки, скобки и вложенность повторяются много раз. На больших массивах это превращается в лишние токены, деньги и latency.

10.2. Пример

```
JSON:
[
{"id": 1, "name": "Alice", "role": "admin"},
{"id": 2, "name": "Bob", "role": "user"}
]
```

```
TOON-подобная идея:
users[2]{id,name,role}:
1,Alice,admin
2,Bob,user
```

10.3. Практическое правило

- Для внешних API, строгих контрактов и structured output чаще оставайтесь на JSON / JSON Schema, если этого требует инструмент.
- Для передачи больших однотипных таблиц или массивов внутрь prompt можно экспериментировать с TOON-like форматами.
- Экономия зависит от структуры данных. Чем больше однотипных объектов с повторяющимися ключами, тем больше потенциальный выигрыш.
- Не считайте TOON универсальной заменой JSON. Это скорее prompt-формат для экономии токенов, а не обязательный стандарт приложения.

11. Цель недели и вариант реализации задания

Цель недели по лекции: познакомиться с context management, memory management и построить простой flow работы агента. То есть не просто вызвать LLM, как на первой неделе, а сделать маленькую систему, где есть состояние, память, планирование и хотя бы один tool.

11.1. Что можно сделать как MVP

Для твоего стека лучше всего делать маленький Python-проект: “Agent Flow Playground”. Это не должен быть большой продукт. Главное - показать, что агент проходит понятный цикл и управляет контекстом/памятью.

```
agent-flow-playground/  
├── README.md  
├── .env.example  
├── requirements.txt  
├── main.py  
├── agent/  
│   ├── __init__.py  
│   ├── llm_client.py  
│   ├── context.py  
│   ├── memory.py  
│   ├── planner.py  
│   ├── tools.py  
│   └── state.py  
├── data/  
│   ├── memory.json  
│   └── runs/  
├── prompts/  
├── system.md  
├── planner.md  
└── summarizer.md
```

11.2. Минимальный flow агента

```
1. User input  
2. Load memory  
3. Build context  
4. Ask LLM to create a plan  
5. Execute one or more tools  
6. Interpret tool result  
7. Generate final answer  
8. Save useful facts / summary / state
```

11.3. Пример tools для простого задания

- calculator_tool: считает простые выражения.
- file_search_tool: ищет по локальным notes/project files.
- save_memory_tool: сохраняет факт или summary в data/memory.json.
- read_memory_tool: достаёт релевантные факты по keyword или простому similarity.
- run_python_tool: опционально запускает безопасный локальный Python-код для проверки.

11.4. Что показать в видео

- Агент стартует без памяти или с пустой memory.
- Пользователь даёт задачу.
- Агент строит план, а не сразу отвечает.
- Агент вызывает tool.
- Агент сохраняет факт/summary в memory.json.

- При следующем запуске агент подгружает сохранённую память.
- В README показано, как запустить проект и какие параметры можно менять.

12. Чек-лист для README и видео-демо

12.1. README

- Название проекта и цель: Week 2 Agent Flow Playground.
- Какие блоки реализованы: context, memory, planning, tools, state.
- Как установить зависимости.
- Как заполнить .env.
- Как запустить demo.
- Где лежит память агента.
- Какие tools доступны.
- Примеры запросов.
- Что ограничено в MVP и что можно улучшить дальше.

12.2. Видео

Сценарий видео на 2-4 минуты:

1. Показать структуру проекта.
2. Показать README и .env.example.
3. Запустить agent demo.
4. Дать задачу агенту.
5. Показать план.
6. Показать tool call и результат.
7. Показать сохранение memory/state.
8. Перезапустить и показать, что память подгрузилась.
9. Коротко объяснить: это context management + memory management + planning + tool orchestration.

13. Готовые промпты для Cursor / Claude Code

13.1. Промпт на создание проекта

Ты senior Python backend engineer и AI agent engineer.

Создай учебный проект для AI Advent Challenge Week 2: Agent Flow Playground.

Цель проекта - показать простой агентный flow: context management, memory management, planning, tool orchestration и state update.

Стек:

- Python 3.11+
- OpenAI-compatible Chat Completions API через httpx
- python-dotenv
- pydantic для структур данных
- JSON-файл как простая long-term memory

Сделай структуру:

agent/llm_client.py, agent/context.py, agent/memory.py, agent/planner.py, agent/tools.py, agent/state.py, main.py, README.md, .env.example, requirements.txt.

Требования:

1. Агент должен принимать user input.
2. Подгружать memory.json.
3. Строить context из system prompt + memory + текущей задачи.

4. Просить LLM сформировать план.
5. Выполнять минимум один tool: calculator или save_memory.
6. Сохранять summary результата в memory.json.
7. Логировать этапы: perception, memory, planning, action, feedback/state update.
8. README должен содержать команды запуска и сценарий демо-видео.

Сначала покажи план реализации и список файлов, затем создай код.

13.2. Промпт на ревью проекта

Проверь мой Week 2 Agent Flow Playground как ревьюер AI Advent Challenge.

Оцени по критериям:

1. Есть ли context management.
2. Есть ли memory management.
3. Есть ли planning stage.
4. Есть ли tool orchestration.
5. Есть ли сохранение state.
6. Понятно ли это показать на видео.
7. Достаточно ли README для сдачи.

Не переписывай всё с нуля. Сначала дай список проблем по приоритетам, потом предложи минимальные правки.

13.3. Промпт на подготовку видео-демо

Составь короткий сценарий демо-видео для сдачи AI Advent Week 2.

Проект: Agent Flow Playground на Python.

Видео должно показать: запуск, user input, context loading, memory loading, planning, tool call, state update, сохранение memory и повторный запуск с подгрузкой памяти.

Сделай сценарий на 2-4 минуты: что говорить и что показывать на экране.

Итог

Лекция 2 переводит фокус с “как спросить LLM” на “как построить управляемого агента”. Для сдачи недели важно показать не красоту кода, а саму механику: агент получает задачу, собирает контекст, использует память, строит план, вызывает инструмент, интерпретирует результат и обновляет state.

Практически лучший путь - сделать небольшой Python playground, где каждая стадия явно логируется и хорошо видна в видео. Тогда проверяющему будет сразу понятно, что ты понял тему недели и можешь применить её в коде.